1] Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
from sklearn.preprocessing import StandardScaler
```

2] Load Dataset

```
df = pd.read_csv('Used Car Dataset.csv')
```

3] Proper Data Printing & Inspection

```
print("Shape:", df.shape)
print(df.head())
print(df.info())
print(df.describe())
```

```
Shape: (1553, 15)
   Unnamed: 0                                           car_name  \
0           0                       2017 Mercedes-Benz S-Class S400
1           1  2020 Nissan Magnite Turbo CVT XV Premium Opt BSVI
2           2                          2018 BMW X1 sDrive 20d xLine
3           3                              2019 Kia Seltos GTX Plus
4           4                       2019 Skoda Superb LK 1.8 TSI AT

  registration_year insurance_validity fuel_type  seats  kms_driven  \
0            Jul-17      Comprehensive    Petrol      5       56000
1            Jan-21      Comprehensive    Petrol      5       30615
2            Sep-18      Comprehensive    Diesel      5       24000
3            Dec-19      Comprehensive    Petrol      5       18378
4            Aug-19      Comprehensive    Petrol      5       44900

    ownsership transmission manufacturing_year  mileage(kmpl)
engine(cc)  \
0  First Owner    Automatic               2017           7.81
2996.0
1  First Owner    Automatic               2020          17.40
999.0
2  First Owner    Automatic               2018          20.68
1995.0
3  First Owner       Manual               2019          16.50
1353.0
```

```
4  First Owner    Automatic                2019          14.67
1798.0

   max_power(bhp)  torque(Nm)  price(in lakhs)
0          2996.0       333.0            63.75
1           999.0      9863.0             8.99
2          1995.0       188.0            23.75
3          1353.0     13808.0            13.56
4          1798.0     17746.0            24.00
<class 'pandas.DataFrame'>
RangeIndex: 1553 entries, 0 to 1552
Data columns (total 15 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Unnamed: 0           1553 non-null   int64
 1   car_name             1553 non-null   str
 2   registration_year    1553 non-null   str
 3   insurance_validity   1553 non-null   str
 4   fuel_type            1553 non-null   str
 5   seats                1553 non-null   int64
 6   kms_driven           1553 non-null   int64
 7   ownsership           1553 non-null   str
 8   transmission         1553 non-null   str
 9   manufacturing_year   1553 non-null   str
 10  mileage(kmpl)        1550 non-null   float64
 11  engine(cc)           1550 non-null   float64
 12  max_power(bhp)       1550 non-null   float64
 13  torque(Nm)           1549 non-null   float64
 14  price(in lakhs)      1553 non-null   float64
dtypes: float64(5), int64(3), str(7)
memory usage: 182.1 KB
None
        Unnamed: 0         seats     kms_driven  mileage(kmpl)
engine(cc)  \
count  1553.000000   1553.000000    1553.000000    1550.000000
1.550000e+03
mean    776.000000     91.480361   52841.931101     236.927277
1.471857e+10
std     448.456798   2403.424060   40067.800347     585.964295
2.185629e+11
min       0.000000      4.000000     620.000000       7.810000
5.000000e+00
25%     388.000000      5.000000   30000.000000      16.342500
1.197000e+03
50%     776.000000      5.000000   49134.000000      18.900000
1.462000e+03
75%    1164.000000      5.000000   70000.000000      22.000000
1.995000e+03
max    1552.000000  67000.000000  810000.000000    3996.000000
```

```
3.258640e+12
```

	max_power(bhp)	torque(Nm)	price(in lakhs)
count	1.550000e+03	1.549000e+03	1553.000000
mean	1.471857e+10	1.423989e+04	166.141494
std	2.185629e+11	9.666241e+04	3478.855090
min	5.000000e+00	5.000000e+00	1.000000
25%	1.197000e+03	4.000000e+02	4.660000
50%	1.462000e+03	1.173000e+03	7.140000
75%	1.995000e+03	8.850000e+03	17.000000
max	3.258640e+12	1.464800e+06	95000.000000

4] Data Cleaning

```python
# ------------------------------------------------------
# 1. FIX COLUMN NAME SPELLING
# ------------------------------------------------------

df = df.rename(columns={'ownsership': 'ownership'})


# ------------------------------------------------------
# 2. REMOVE REDUNDANT COLUMN
# ------------------------------------------------------

if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])


# ------------------------------------------------------
# 3. HANDLE MISSING VALUES (PROPER WAY)
# ------------------------------------------------------

print("\nMissing Values Before:")
print(df.isnull().sum())

# Select numeric and categorical columns properly
num_cols = df.select_dtypes(include=[np.number]).columns
cat_cols = df.select_dtypes(include=['object', 'string']).columns

# Fill numeric columns with median (NO inplace, NO chained assignment)
for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill categorical columns with mode
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print("\nMissing Values After:")
print(df.isnull().sum())
```

```python
# ------------------------------------------------------
# 4. REMOVE DUPLICATES
# ------------------------------------------------------

print("\nDuplicate Rows Before:", df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicate Rows After:", df.duplicated().sum())


# ------------------------------------------------------
# 5. CLEAN YEAR COLUMNS
# ------------------------------------------------------

if 'registration_year' in df.columns:
    df['registration_year'] = (
        df['registration_year']
        .astype(str)
        .str[-2:]
    )
    df['registration_year'] = pd.to_numeric(
        df['registration_year'], errors='coerce'
    ) + 2000

if 'manufacturing_year' in df.columns:
    df['manufacturing_year'] = pd.to_numeric(
        df['manufacturing_year'], errors='coerce'
    )

# Fill any NaN created after conversion
for col in ['registration_year', 'manufacturing_year']:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())


# ------------------------------------------------------
# 6. REMOVE OUTLIERS USING IQR
# ------------------------------------------------------

num_cols = df.select_dtypes(include=[np.number]).columns

Q1 = df[num_cols].quantile(0.25)
Q3 = df[num_cols].quantile(0.75)
IQR = Q3 - Q1

mask = ~((df[num_cols] < (Q1 - 1.5 * IQR)) |
         (df[num_cols] > (Q3 + 1.5 * IQR))).any(axis=1)

df = df[mask]
```

```
print("\nFinal Shape After Cleaning:", df.shape)

print("\nData Cleaning Completed Successfully ")
```

```
Missing Values Before:
car_name               0
registration_year      0
insurance_validity     0
fuel_type              0
seats                  0
kms_driven             0
ownership              0
transmission           0
manufacturing_year     0
mileage(kmpl)          2
engine(cc)             2
max_power(bhp)         2
torque(Nm)             2
price(in lakhs)        0
dtype: int64

Missing Values After:
car_name               0
registration_year      0
insurance_validity     0
fuel_type              0
seats                  0
kms_driven             0
ownership              0
transmission           0
manufacturing_year     0
mileage(kmpl)          0
engine(cc)             0
max_power(bhp)         0
torque(Nm)             0
price(in lakhs)        0
dtype: int64

Duplicate Rows Before: 0
Duplicate Rows After: 0

Final Shape After Cleaning: (615, 14)

Data Cleaning Completed Successfully 
```

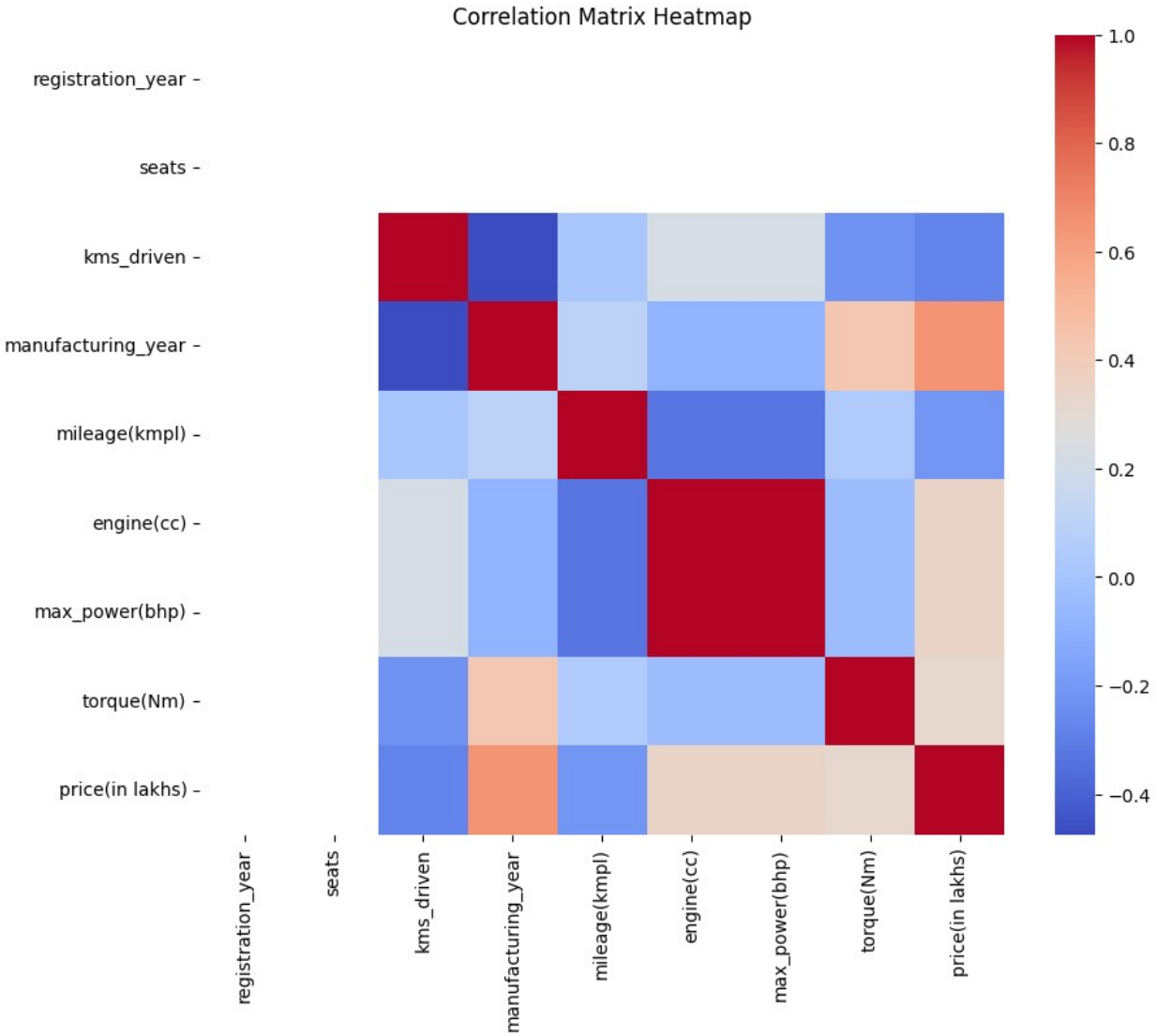
5] Feature Selection Using Correlation

```
# ======================================================
# FEATURE SELECTION USING CORRELATION
```

```
# ========================================================

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

target = 'price(in lakhs)'

# --------------------------------------------------------
# 1. COMPUTE CORRELATION MATRIX (NUMERIC ONLY)
# --------------------------------------------------------

corr_matrix = df.corr(numeric_only=True)

print("Correlation with Target Variable:\n")
print(corr_matrix[target].sort_values(ascending=False))


# --------------------------------------------------------
# 2. VISUALIZE CORRELATION HEATMAP
# --------------------------------------------------------

plt.figure(figsize=(10,8))
sns.heatmap(corr_matrix, cmap='coolwarm', annot=False)
plt.title("Correlation Matrix Heatmap")
plt.show()


# --------------------------------------------------------
# 3. SELECT IMPORTANT FEATURES
#    (Using Threshold Method)
# --------------------------------------------------------

threshold = 0.3   # You can adjust (0.2–0.5 depending on strictness)

important_features = corr_matrix[target][
    abs(corr_matrix[target]) > threshold
].index.tolist()

print("\nFeatures Selected Based on Correlation Threshold:")
print(important_features)


# --------------------------------------------------------
# 4. REMOVE TARGET FROM FEATURE LIST
# --------------------------------------------------------

important_features.remove(target)

X = df[important_features]
```

```python
y = df[target]

print("\nFinal Selected Feature Columns:")
print(X.columns)


# ----------------------------------------------------
# 5. CHECK MULTICOLLINEARITY (OPTIONAL BUT IMPORTANT)
#    Remove highly correlated independent variables
# ----------------------------------------------------

feature_corr = X.corr().abs()

upper_triangle = feature_corr.where(
    np.triu(np.ones(feature_corr.shape), k=1).astype(bool)
)

# Drop features with correlation > 0.8
to_drop = [column for column in upper_triangle.columns
           if any(upper_triangle[column] > 0.8)]

print("\nHighly Correlated Features to Drop (Multicollinearity):")
print(to_drop)

X = X.drop(columns=to_drop)

print("\nFinal Features After Removing Multicollinearity:")
print(X.columns)
```

```
Correlation with Target Variable:

price(in lakhs)       1.000000
manufacturing_year    0.654323
engine(cc)            0.347797
max_power(bhp)        0.347797
torque(Nm)            0.318608
mileage(kmpl)        -0.215443
kms_driven           -0.283131
registration_year          NaN
seats                      NaN
Name: price(in lakhs), dtype: float64
```




```
Features Selected Based on Correlation Threshold:
['manufacturing_year', 'engine(cc)', 'max_power(bhp)', 'torque(Nm)',
'price(in lakhs)']

Final Selected Feature Columns:
Index(['manufacturing_year', 'engine(cc)', 'max_power(bhp)',
'torque(Nm)'], dtype='str')

Highly Correlated Features to Drop (Multicollinearity):
['max_power(bhp)']

Final Features After Removing Multicollinearity:
Index(['manufacturing_year', 'engine(cc)', 'torque(Nm)'], dtype='str')
```

6] Encode Categorical Variables

```
X = pd.get_dummies(X, drop_first=True)
```

7] Feature Scaling (Z-Score)

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

8] Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,
    random_state=42
)
```

9] Train Multiple Linear Regression Model

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression()
```

10] View Model Parameters

```
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

```
Intercept: 6.187558264961474
Coefficients: [1.80053186 1.05593927 0.14246932]
```

11] Make Predictions

```
y_pred = model.predict(X_test)
```

12] Evaluate Model

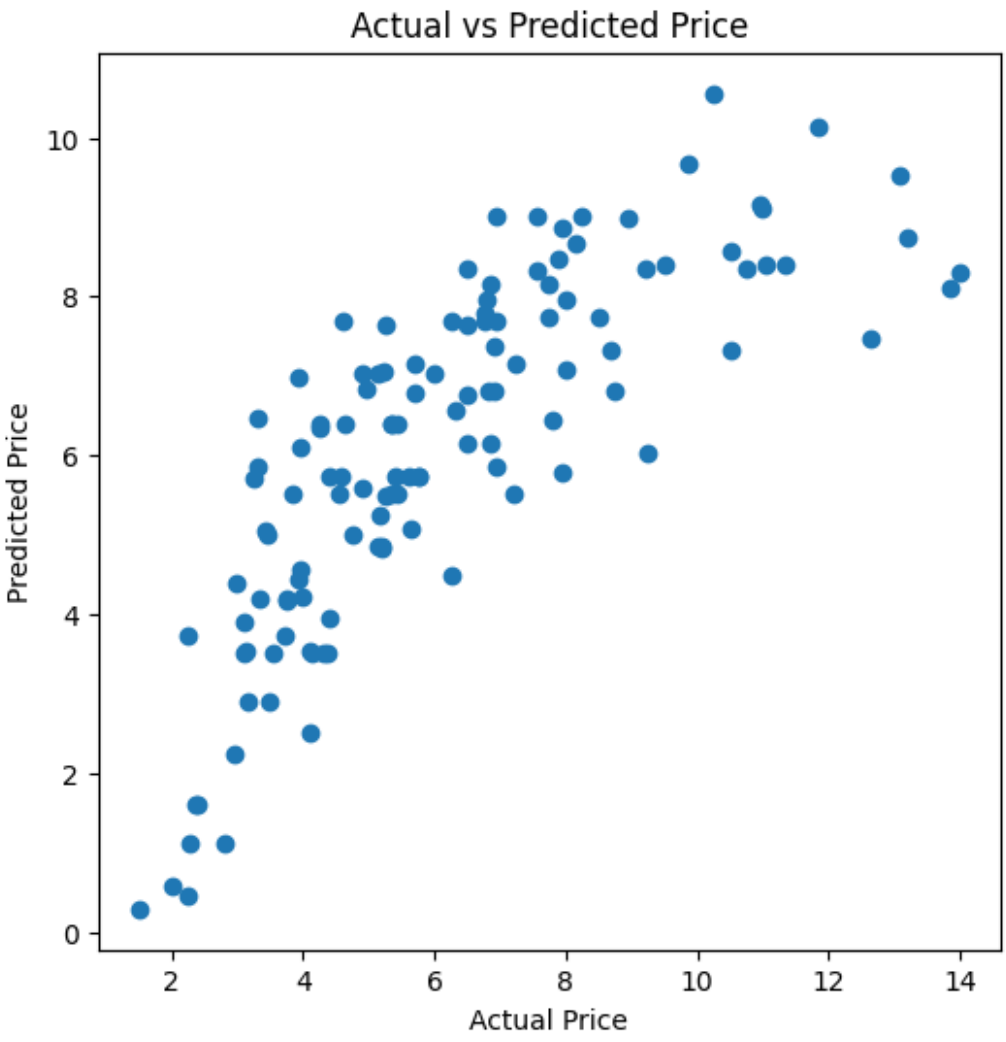
```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R² Score:", r2)
```

```
MAE: 1.2620698936411472
MSE: 2.846379666697234
RMSE: 1.6871217106946477
R² Score: 0.6240008030524731
```

13] Visualization

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Price")
plt.show()
```


Actual vs Predicted Price
Predicted Price
Actual Price
0
2
4
6
8
10
2
4
6
8
10
12
14